

Vector Stores in LangChain

1. Introduction

When building AI apps (like “Chat with PDF”), we need to:

- Store chunks of text
- Search them efficiently
- Retrieve the most relevant ones

👉 Normal databases are **not good at this**

👉 That's why we use **Vector Stores**

2. Why Vector Stores?

🚨 Problem:

Suppose you have 1000 text chunks.

User asks:

"What is machine learning?"

❌ Traditional search (keyword-based):

- Looks for exact words
 - Misses meaning
-

Example:

Text:

"ML is a subset of AI"

Search:

"What is machine learning?"

👉 Keyword search may fail ❌

✅ Vector Search:

- Understands meaning (semantics)
- Finds related content

👉 Even if words are different ✓

💡 Key Idea:

Instead of matching words → match **meaning**

3. What are Vector Stores?

✅ Simple Definition:

A Vector Store:

Stores text as **vectors (numbers)** and allows similarity search

💡 What is a Vector?

A vector is:

A list of numbers representing meaning

Example:

Text:

"Machine learning is part of AI"

Vector:

[0.12, -0.45, 0.88, ...]

👉 This vector captures the **semantic meaning**

🔄 Process:

- ❖ Text → Embedding model
 - ❖ Embedding → Vector
 - ❖ Store in vector store
-

🧠 How Search Works

Step-by-step:

- User query:

"What is AI?"

- Convert to vector
 - Compare with stored vectors
 - Find **most similar vectors**
-

👉 Return relevant chunks

4. Vector Store vs Vector Database

⚠ Important Difference:

Feature	Vector Store	Vector Database
Definition	Tool/library	Full database system

Scope	Used inside apps	Standalone system
Example	LangChain Chroma	Pinecone, Weaviate

Simple Analogy:

- Vector Store = calculator
 - Vector Database = full computer system
-

In LangChain:

We mostly use **vector stores as components**

5. Vector Stores in LangChain

LangChain provides built-in support for vector stores.

Common Options:

- Chroma
 - FAISS
 - Pinecone
 - Weaviate
-

These help:

- Store embeddings
 - Perform similarity search
 - Retrieve documents
-

Full Flow (Very Important)

Text → Split → Embedding → Vector Store → Retrieval → LLM

Vector Store is the **memory system**

6. Chroma Vector Store & Code

What is Chroma?

Chroma is:

A simple and popular vector store (easy for beginners)

Example Code:

```
from langchain.vectorstores import Chroma
from langchain.embeddings import OpenAIEmbeddings
```

```
# Create embeddings
embedding = OpenAIEmbeddings()

# Create vector store
vectorstore = Chroma.from_documents(docs, embedding)
```

Searching:

```
results = vectorstore.similarity_search("What is AI?")
```



What happens:

1. Query → vector
2. Compare with stored vectors
3. Return most similar chunks

Output:

Relevant text chunks from your data



Real-World Example

Task: Chat with PDF

Step-by-step:

1. Load PDF
2. Split into chunks
3. Convert to embeddings
4. Store in vector store
5. User asks question
6. Retrieve relevant chunks
7. Send to LLM
8. Generate answer



Vector Store makes retrieval **fast and smart**



Important Concepts



1. Similarity Search

Find closest meaning, not exact words

2. Embeddings

Convert text → numbers

3. Retrieval

Get most relevant chunks

4. Top-K Results

Return top N similar chunks

Common Mistakes

1. Using large chunks → poor results
 2. No overlap → context loss
 3. Wrong embedding model
 4. Expecting keyword matching
-

Big Picture

Without Vector Store:

-  Cannot search meaning
 -  Poor answers
-

With Vector Store:

-  Smart retrieval
 -  Better context
 -  Accurate answers
-

Final Summary

- **Vector Store = stores text as vectors**
 - Used for **semantic search**
 - Key steps:
 - Text → Embeddings → Store → Search
 - Difference:
 - Vector Store = tool
 - Vector DB = full system
 - Popular in LangChain:
 - Chroma (beginner-friendly)
-

One-Line Understanding

👉 **Vector Store = brain memory that finds information based on meaning, not keywords**
